

Bataille Navale

SOMMAIRE

Etape 1 :

Etape 2 :

Etape 3 :

Etape 4 :

Etape 5 :

Etape 6 :

Etape 7 :

Etape 8 :

Etape 1 :

Dans cette première étape, on déclare deux tableaux à deux dimensions : tabjour et tabordi. Chaque tableau est créé avec la ligne suivante :

```
int tabjour[][] = new int[5][5];  
int tabordi[][] = new int[5][5];
```

Ensuite, on initialise toutes les cases du tableau avec la valeur 0 à l'aide de deux boucles imbriquées. La première boucle parcourt les lignes, et la seconde parcourt les colonnes. À chaque itération, la valeur 0 est attribuée à la case correspondante :

```
for (int i = 0; i < tabjour.length; i++) {  
    for (int j = 0; j < tabjour[i].length; j++) {  
        tabjour[i][j] = 0;  
        tabordi[i][j] = 0;  
    }  
}
```

Etape 2 :

On définit le nombre total de pions à placer et un compteur pour savoir combien ont déjà été posés. La boucle while continue tant que tous les pions n'ont pas été placés.

```
// Étape 2 : placement des pions du joueur  
int nbPions = 5;  
int pionsPlaces = 0;  
  
while (pionsPlaces < nbPions) {  
    System.out.println("\nPlacement du pion n°" + (pionsPlaces + 1));
```

2.1

Le programme demande à l'utilisateur la ligne où il veut placer son pion et lit la valeur saisie.

```
// 2.1 demander la ligne  
System.out.print("Choisis la ligne (0 à 4) : ");  
int ligne = sc.nextInt();
```

2.2

Le programme demande à l'utilisateur la colonne où il veut placer son pion et lit la valeur saisie.

```
// 2.2 demander la colonne
System.out.print("Choisis la colonne (0 à 4) : ");
int colonne = sc.nextInt();
```

2.3

Le code vérifie si la ligne et la colonne sont dans les limites du tableau et si la case est libre. Si oui, le pion est placé et le compteur augmente.

```
// 2.3 vérifications et placement
if (ligne < 0 || ligne >= 5 || colonne < 0 || colonne >= 5) {
    System.out.println("Erreur : position en dehors du tableau !");
} else if (tabjoueur[ligne][colonne] == 1) {
    System.out.println("Erreur : cette case contient déjà un pion !");
} else {
    tabjoueur[ligne][colonne] = 1;
    pionsPlaces++;
    System.out.println("Pion placé en (" + ligne + ", " + colonne + ")");
}
```

Résultat :

```
Placement du pion n°1
Choisis la ligne (0 à 4) : 1
Choisis la colonne (0 à 4) : 2
Pion placé en (1, 2)
```

```
Placement du pion n°2
Choisis la ligne (0 à 4) : 1
Choisis la colonne (0 à 4) : 2
Erreur : cette case contient déjà un pion !
```

```
Placement du pion n°2
Choisis la ligne (0 à 4) : 5
Choisis la colonne (0 à 4) : 9
Erreur : position en dehors du tableau !
```

Etape 3 :

On initialise un compteur pour savoir combien de pions l'ordinateur a déjà mis.

```
int pionsOrdiPlaces = 0;
```

3.1/3.2

On prend une ligne et une colonne au hasard entre 0 et 4. Ces coordonnées correspondent à une case sur le tableau de l'ordinateur.

```
//3.1 et 3.2 attribuer une ligne aléatoirement et une colonne  
int ligneOrdi = (int)(Math.random() * 5);  
int colonneOrdi = (int)(Math.random() * 5);
```

3.3

Si la case est vide (0), on place un pion (1) et on augmente le compteur. Sinon, on recommence avec de nouvelles coordonnées.

```
if (tabordi[ligneOrdi][colonneOrdi] == 0) {  
    tabordi[ligneOrdi][colonneOrdi] = 1;  
    pionsOrdiPlaces++;  
}
```

Etape 4 :

Cette procédure prend un tableau à deux dimensions en entrée. Elle affiche d'abord les numéros des colonnes en haut. Ensuite, pour chaque ligne, elle affiche le numéro de ligne suivi des symboles des cases : ~ pour vide et O pour un pion.

```

System.out.print(" ");
for (int j = 0; j < taille; j++) {
    System.out.print(j + 1 + " ");
}
System.out.println();

for (int i = 0; i < taille; i++) {
    System.out.print(i + 1 + " ");
    for (int j = 0; j < taille; j++) {
        if (tab[i][j] == 0) {
            System.out.print("~ ");
        } else if (tab[i][j] == 1) {
            System.out.print("O ");
        }
    }
    System.out.println();
}

```

On appelle la procédure pour afficher la grille du joueur. On voit où chaque pion a été placé et quelles cases restent vides.

```

// Affichage des grilles
System.out.println("\nGrille du joueur :");
afficherGrille(tabjoueur);

```

Résultat :

```

Grille du joueur :
  1 2 3 4 5
1 0 ~ ~ ~ ~
2 ~ 0 ~ ~ ~
3 ~ ~ 0 ~ ~
4 ~ ~ ~ 0 ~
5 ~ ~ ~ ~ 0

```

Etape 5 :

```
System.out.print(" ");
for (int i = 1; i <= nbcase; i++) {
    System.out.print(i + " ");
}
System.out.println();
// Pour chaque ligne
for (int i = 0; i < nbcase; i++) {
    System.out.print((i + 1) + " ");
    for (int j = 0; j < nbcase; j++) {
        if (tab[i][j] == 0) {
            System.out.print("? ");
        }
        else if (tab[i][j] == 1) {
            System.out.print("? ");
        }
        else if (tab[i][j] == 2) {
            System.out.print("o ");
        }
        else if (tab[i][j] == 3) {
            System.out.print("x ");
        }
    }
    System.out.println();
}
}
```

Pour l'étape 5, j'ai fait une fonction nommée `affichagetabCache` qui gère l'affichage de la grille adverse en masquant les navires non découverts. Le principe est simple : les cases contenant 0 ou 1 sont représentées par un point d'interrogation pour maintenir le suspense. Lorsqu'une case contient la valeur 2, cela signifie qu'un tir a atteint sa cible, donc j'utilise le symbole 'o'. Pour les valeur 3, qui correspondent aux tirs manqués, j'affiche 'x'. Ce système empêche les joueurs de tricher en consultant l'emplacement réel des bateaux adverses

Résultat :

```
Choisis la ligne (0 a 4) : 2
Choisis la colonne (0 a 4) : 3
Cette case contient deja un pion !
Choisis la ligne (0 a 4) : 4
Choisis la colonne (0 a 4) : 1
Pion place en (4, 1)
Grille du joueur :
1 2 3 4 5
1 ~ ~ ~ ~ ~
2 ~ ~ ~ ~ ~
3 ~ ~ o o ~
4 ~ ~ o o ~
5 ~ o ~ ~ ~
Grille de l'ordinateur :
1 2 3 4 5
1 o ~ ~ ~ ~
2 o ~ ~ o ~
3 ~ ~ ~ ~ o
4 ~ ~ ~ ~ ~
5 ~ ~ ~ ~ o
```

Etape 6 :

6.1: Demandez à l'utilisateur a quelle ligne il veut découvrir une case

```
System.out.print("Choisis la ligne (0 a 4) : ");
int ligneTir = sc.nextInt();
```

6.2 : Demandez à l'utilisateur a quelle colonne il veut découvrir une case

```
System.out.print("Choisis la colonne (0 a 4) : ");
int colonneTir = sc.nextInt();
```

6.3 : Afficher "Tir en cours" et laisser une attente de 2 secondes pour "simuler" le tir

```

System.out.println("Tir en cours...");
try {
    Thread.sleep(2000);
} catch (InterruptedException e) {
    e.printStackTrace();
}

```

6.4 Afficher le résultat

Si la case était sans pion (valeur 0) : il faut afficher rate !

```

if (tabOrdi[ligneTir][colonneTir] == 0) {
    System.out.println("Rate !");
    tabOrdi[ligneTir][colonneTir] = 3; // 3 = case rate
}

```

Si la case était avec pion (valeur 1) : afficher Touche ! + incrémenter la variable nbPionTrouveJoueur

```

else if (tabOrdi[ligneTir][colonneTir] == 1) {
    System.out.println("Touche !");
    tabOrdi[ligneTir][colonneTir] = 2; // 2 = pion touche
    nbPionTrouveJoueur++;
}

```

Toutes autres valeurs dans la case cela signifie que le joueur avait déjà découvert cette case : afficher Tir a blanc !

```

else {
    System.out.println("Tir a blanc !");
}

```

6.5 Afficher le tableau ordi : Faites juste un appel a la procedure cree en etape 5

```

System.out.println("\nGrille de l'ordinateur:");
affichageCache(tabOrdi, 5);
sc.close();
}

```

Le tour du joueur est celui de la phase de tir. Pour commencer, on affiche "À vous de jouer", invitant le joueur à saisir successivement la ligne puis la colonne de son choix à l'aide d'un Scanner. Afin de simuler le tir de façon plus réaliste, on affiche ensuite "Tir en cours" et le

programme effectue une pause de deux secondes grâce à Thread.sleep. C'est après cette pause que l'on vérifie l'état de la case ciblée : si elle contient la valeur 0, le tir est un "Raté", et on lui assigne la valeur 3. Si elle contient la valeur 1, le tir est "Touché", on la remplace par la valeur 2 et on augmente le compte des bateaux découverts. Enfin, si la case avait déjà les valeurs 2 ou 3, on annonce un "Tir à blanc" puisque la zone a déjà été ciblée. Le tour se conclut en appelant la procédure de l'étape 5 pour afficher la grille mise à jour, tout en masquant les positions encore inconnues.

Etape 7 :

7.0 Afficher "Tour de l'ordinateur"

```
System.out.println("\n=====");
System.out.println("Tour de l'ordinateur");
System.out.println("=====");
boolean tirValide = false;
while (!tirValide) {
```

7.1 Attribuer une ligne aléatoirement (entre 1 et 5)

```
int ligneTirOrdi = (int)(Math.random() * 5);
```

7.2 Attribuer une colonne aléatoirement (entre 1 et 5)

```
int colonneTirOrdi = (int)(Math.random() * 5
```

7.3 Attention l'ordinateur ne doit pas choisir une case qui a été découverte (valeur autre que 0 ou 1)

```
if (tabJoueur[ligneTirOrdi][colonneTirOrdi] == 0 || tabJoueur[ligneTirOrdi][colonneTirOrdi] == 1) {
    tirValide = true;
```

7.4 Afficher "Tir en cours" et laisser une attente de 2 secondes pour "simuler" le tir

```
System.out.println("Tir en cours...");
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    e.printStackTrace();
}
```

7.5 Afficher le resultat

Si la case était sans pion (valeur 0) : afficher Rate !

```

if (tabJoueur[ligneTirOrdi][colonneTirOrdi] == 0) {
    System.out.println("L'ordinateur a rate !");
    tabJoueur[ligneTirOrdi][colonneTirOrdi] = 3; // 3 = case rate
}

```

Si la case etait avec pion (valeur 1) : afficher Touche ! + incrémenter la variable nbPionTrouveOrdi

```

    else if (tabJoueur[ligneTirOrdi][colonneTirOrdi] == 1) {
        System.out.println("L'ordinateur a touche !");
        tabJoueur[ligneTirOrdi][colonneTirOrdi] = 2; // 2 = pion touche
        nbPionTrouveOrdi++;
        System.out.println("L'ordinateur a trouve " + nbPionTrouveOrdi + "/5 bateaux");
    }
}
}

```

7.6 Afficher le tableau joueur : Faites juste un appel a la procedure cree en etape 5

```

System.out.println("\nVotre grille :");
affichageGetabCache(tabJoueur, 5);
}

```

Etape 8 :

Afficher le vainqueur

```

System.out.println("\n=====");
System.out.println("      FIN DE PARTIE");
System.out.println("=====");
if (nbPionTrouveJoueur >= 5) {
    System.out.println("VICTOIRE ! Vous avez gagne !");
} else {
    System.out.println("DEFAITE ! L'ordinateur a gagne !");
}
}

```

Demandez de refaire une partie

```

System.out.print("\nVoulez-vous refaire une partie ? (oui/non) : ");
String reponse = sc.next();
if (reponse.equalsIgnoreCase("oui")) {
    System.out.println("Relancez le programme pour rejouer !");
} else {
    System.out.println("Merci d'avoir joue !");
}
sc.close();
}

```